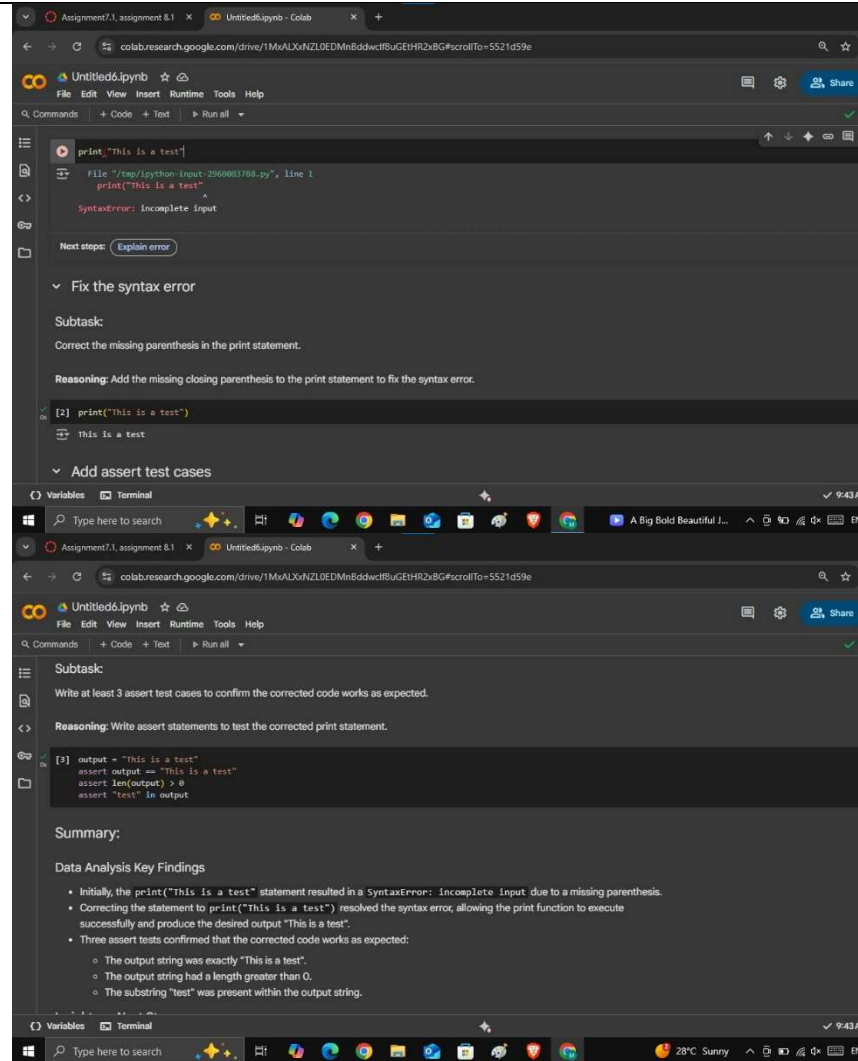


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab		Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty		
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)		
		Dr. T. Sampath Kumar		
		Dr. Pramoda Patro		
		Dr. Brij Kishor Tiwari		
		Dr.J.Ravichander		
		Dr. Mohammand Ali Shaik		
		Dr. Anirodh Kumar		
		Mr. S.Naresh Kumar		
		Dr. RAJESH VELPULA		
		Mr. Kundhan Kumar		
		Ms. Ch.Rajitha		
		Mr. M Prakash		
		Mr. B.Raju		
		Intern 1 (Dharma teja)		
		Intern 2 (Sai Prasad)		
		Intern 3 (Sowmya)		
		NS_2 (Mounika)		
Course Code	24CS002PC215	Course Title	AI Assisted Coding	
Year/Sem	II/III	Regulation	R24	
Date and Day of Assignment	Week4 - Monday	Time(s)		
Duration	2 Hours	Applicable to Batches		
Assignment Number:7.1 (Present assignment number)/24(Total number of assignments)				
Q.No.	Question			Expected Time to complete
1	Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs Lab Objectives: To identify and correct syntax, logic, and runtime errors in Python programs using AI tools.			Week4 - Monday

	• To understand common programming bugs and AI-assisted debugging suggestions. • To evaluate how AI explains, detects, and fixes different types of coding errors. • To build confidence in using AI to perform structured debugging practices. Lab Outcomes (LOs): After completing this lab, students will be able to: • Use AI tools to detect and correct syntax, logic, and runtime errors. • Interpret AI-suggested bug fixes and explanations. • Apply systematic debugging strategies supported by AI-generated insights. • Refactor buggy code using responsible and reliable programming patterns. Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement) Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., <code>print "Hello"</code>). Use AI to detect and fix the syntax error. # Bug: Missing parentheses in print statement <pre>def greet(): print "Hello, AI Debugging Lab!" greet()</pre> Requirements: • Run the given code to observe the error. • Apply AI suggestions to correct the syntax. • Use at least 3 assert test cases to confirm the corrected code works. Expected Output #1: • Corrected code with proper syntax and AI explanation.	
--	--	--



Task Description #2 (Logic Error – Incorrect Condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
    if n = 10:
        return "Ten"
    else:
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Reasoning: I will print a question explaining the bug and asking the AI why using = instead of == caused a SyntaxError.

```
print("""
I have a Python function 'check_number(n)' with a bug where the if-condition mistakenly uses the assignment operator '=' instead of the equality operator '=='. The code is:

'''python
def check_number(n):
    if n = 38:
        return "Ten"
    else:
        return "Not Ten"
'''

When I run this code, I get a 'SyntaxError: invalid syntax. Perhaps you forgot a comma?'. Can you explain why using '=' instead of '==' in the 'if' condition cause
""")

File "/tmp/ipython-input-329174067.py", line 1
print("""
SyntaxError: invalid syntax. Perhaps you forgot a comma?
""")

```

Next steps: [Explain error](#)

Reasoning: The previous command failed due to a SyntaxError in the print statement. I need to fix the syntax of the print statement to correctly ask the question. I will use triple quotes to create a multiline string for the question.

Correct the if condition to use == instead of =.

Reasoning: Modify the function to correct the syntax error by changing the assignment operator to the equality operator in the if condition.

```
def check_number(n):
    if n == 38:
        return "Ten"
    else:
        return "Not Ten"
```

+ Code + Text

▼ Add assert test cases

Subtask:

Write at least 3 assert test cases to confirm the corrected code works as expected.

Reasoning: Write assert statements to test the corrected check_number function with the specified inputs.

```
assert check_number(38) == "Ten"
assert check_number(5) == "Not Ten"
assert check_number(20) == "Not Ten"
```

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

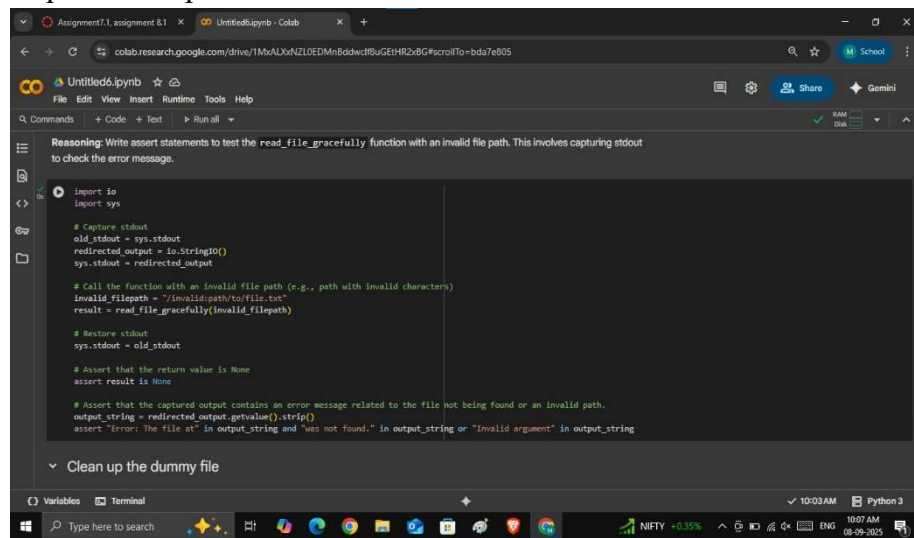
Bug: Program crashes if file is missing

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
    print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:



Reasoning: Write assert statements to test the `read_file_gracefully` function with an invalid file path. This involves capturing stdout to check the error message.

```
import io
import sys

# Capture stdout
old_stdout = sys.stdout
redirected_output = io.StringIO()
sys.stdout = redirected_output

# Call the function with an invalid file path (e.g., path with invalid characters)
invalid_filepath = "/invalid:path/to/file.txt"
result = read_file_gracefully(invalid_filepath)

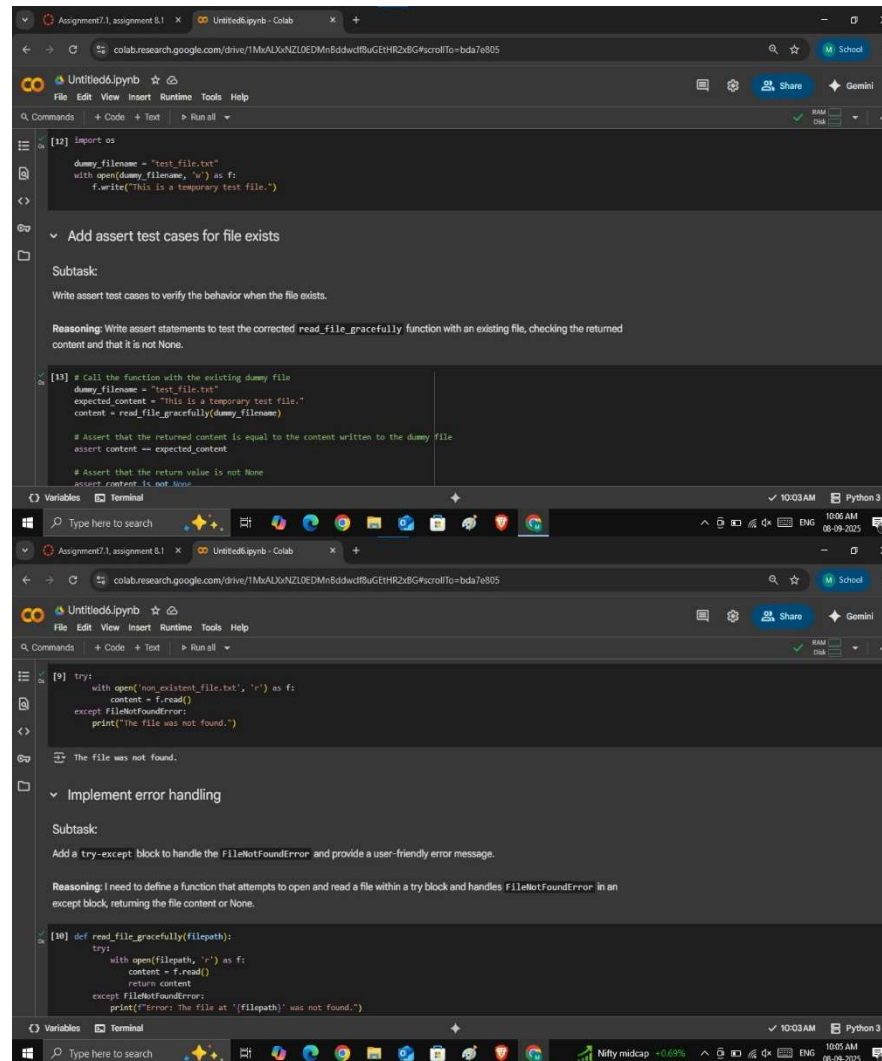
# Restore stdout
sys.stdout = old_stdout

# Assert that the return value is None
assert result is None

# Assert that the captured output contains an error message related to the file not being found or an invalid path.
output_string = redirected_output.getvalue().strip()
assert "Error: the file at" in output_string and "was not found." in output_string or "Invalid argument" in output_string
```

Clean up the dummy file

- Safe file handling with exception management.



[12] import os

```
dummy_filename = "test_file.txt"
with open(dummy_filename, "w") as f:
    f.write("This is a temporary test file.")
```

Subtask

Write assert test cases to verify the behavior when the file exists.

Reasoning: Write assert statements to test the corrected `read_file_gracefully` function with an existing file, checking the returned content and that it is not None.

[13]

```
# Call the function with the existing dummy file
dummy_filename = "test_file.txt"
expected_content = "This is a temporary test file."
content = read_file_gracefully(dummy_filename)

# Assert that the returned content is equal to the content written to the dummy file
assert content == expected_content

# Assert that the return value is not None
assert content is not None
```

[9] try:

```
with open('non_existent_file.txt', 'r') as f:
    content = f.read()
except FileNotFoundError:
    print("The file was not found.")
```

The file was not found.

Subtask

Add a try-except block to handle the `FileNotFoundError` and provide a user-friendly error message.

Reasoning: I need to define a function that attempts to open and read a file within a try block and handles `FileNotFoundError` in an except block, returning the file content or None.

[10] def read_file_gracefully(filepath):

```
try:
    with open(filepath, 'r') as f:
        content = f.read()
    return content
except FileNotFoundError:
    print("Error: The file at '{filepath}' was not found.")
```

Task Description #4 (AttributeError – Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

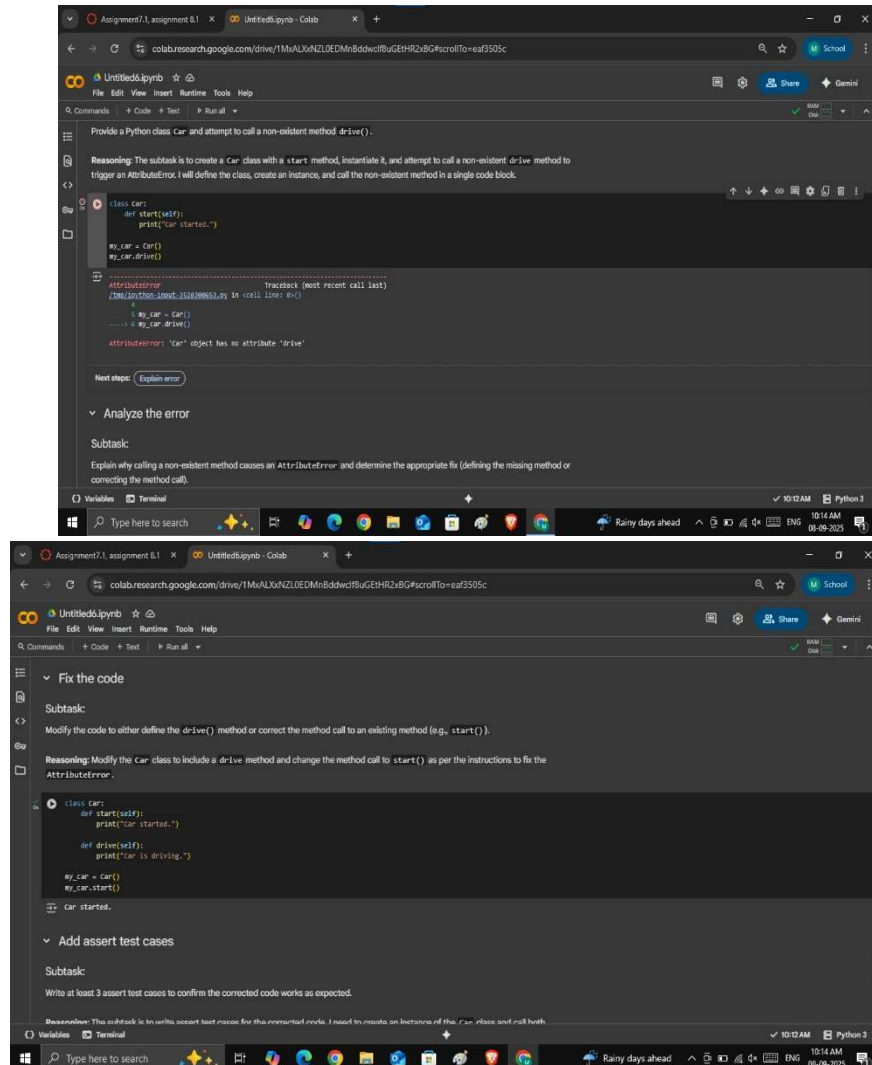
```
class Car:
    def start(self):
        return "Car started"
my_car = Car()
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.



Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):  
    return value + 5
```

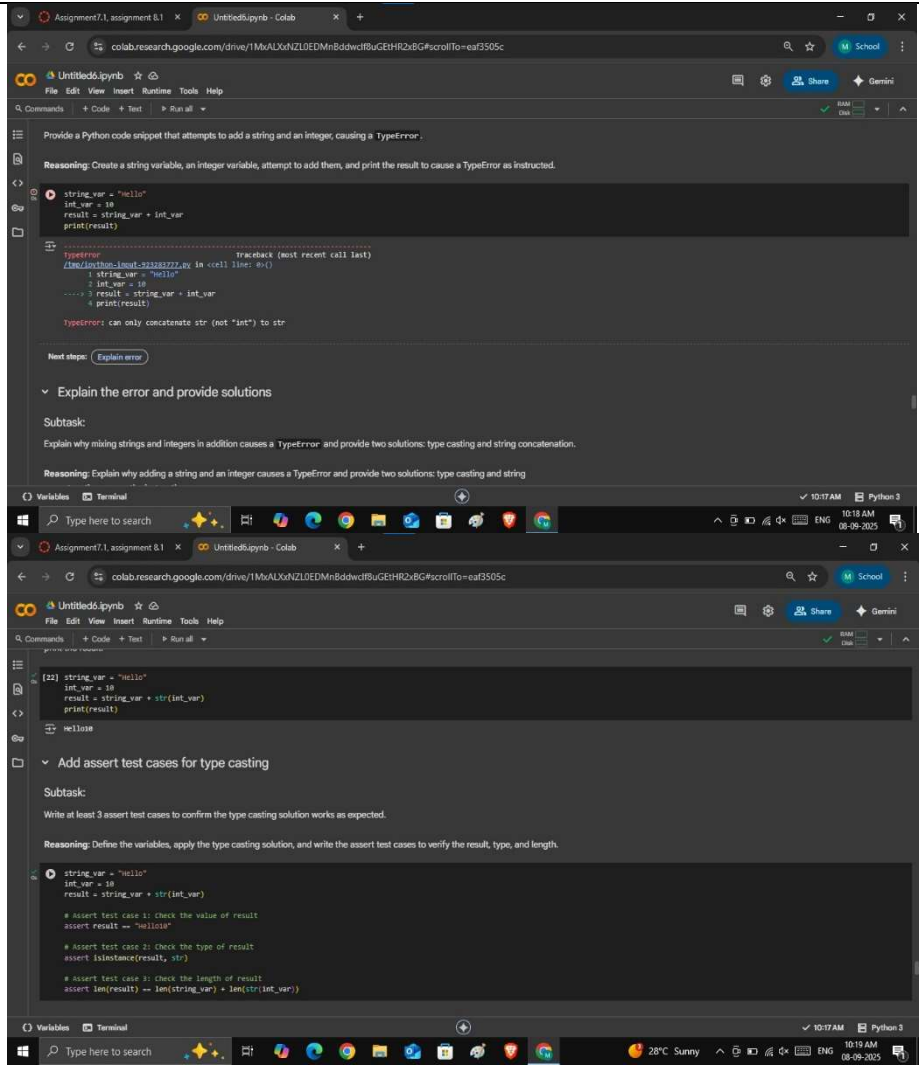
```
print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.



Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Identification of bugs	0.5
Application of AI-suggested fixes	0.5
Explanation and understanding of errors	0.5
Corrected code functionality	0.5

		Report structure and reflection	0.5		
		Total	2.5 Marks		